

Commission & Consolidator Payouts

Module specification for CRM integration

A. Appel and Co · sub-agent under Main St. Travel (IATA 33535983)

This module answers one question automatically, on every ticket: **how much commission is due, and to whom**. It reads the ticket, finds the contract that governs it, computes the figure, and pre-fills the commission field. Once a week the consolidator statement is uploaded and the module reconciles what was actually paid against what was owed, flagging every difference.

It is deliberately built as a **pure function**: text in, JSON out. No database calls, no clock, no network. That is what lets the same code run inside the CRM, in a queue worker, in an Edge Function, and in tests — and it is why the same ticket always produces the same number.

1 · The flow



2 · Contract resolution — the lookup chain

This is the core of the module. Every ticket carries the IATA number it was issued under; that number identifies the consolidator, and the consolidator owns the contracts. The chain is walked **in order**, and stops at the first step that fails.

#	Step	Read from	If it fails
1	Tenant	the CRM session	reject — never price across tenants
2	IATA number	ticket number prefix / agency field in the AIR file	FALLBACK — no contract can be chosen

#	Step	Read from	If it fails
3	Consolidator	contracts table, matched on IATA + tenant	FALLBACK — IATA is not one we hold
4	Carrier contract	validating carrier + issue date inside the contract's validity window	FALLBACK — no contract for this airline
5	Clause	route / origin / booking class / fare type / tour code	NO_RULE — contract exists, no clause covers this ticket
6	Rate	rate table by booking class	NIL or AMBIGUOUS — see §4

Steps 5 and 6 are **not** a fallback. A contract that exists and says *nothing is due* is a different fact from having no contract at all, and the module must never merge them. The first is a settled answer worth writing down; the second is a gap in the data.

The distinction that matters most. NIL means a clause asserted zero — for example, EI AI forfeits commission when the mandatory tour code is missing. NO_RULE means nobody has worked it out yet. Both display as \$0.00. Only NIL may be written into the commission field; NO_RULE must stay empty and go to a queue. Merging them silently books zero on tickets that were actually owed money.

3 · Data model

Five tables. Everything is editable, and contracts are uploadable as files.

tenant

Column	Notes
id, name	Every query below is scoped by tenant_id. Contracts never cross tenants.

consolidator

Column	Notes
id, tenant_id, name	e.g. "Main St Travel"
iata	The join key. Matched against the IATA on the ticket. Unique per tenant.
retains_points	Points of the fare the consolidator keeps; the sub-agent takes the rest.
fee_schedule	Optional. Names a signed fee schedule (see §6). Null ⇒ plain retention only.

carrier_contract

Column	Notes
id, consolidator_id	
carrier	Validating carrier, e.g. LY.
issued_from, issued_to	Validity window, tested against the ticketing date .
rates	JSON: booking class → percentage. {"I":"9.00","S":"7.00", ...}

Column	Notes
include_yq	Whether the commissionable basis is base fare only, or base + YQ/YR.
required_tour_code	If set, a ticket without it earns nothing (asserted NIL).
origin_in	Origination countries the contract covers, e.g. [US, CA].
scope	ticket coupon half_rt — how the rate is applied.
files[]	Uploaded PDFs. Stored for audit; the rules above are what actually run.

ticket_document · statement_row

Table	Notes
ticket_document	One row per passenger per document. Ticket number, type (TKT/EXCH/RFND/VOID), issue date, carrier, IATA, base fare, taxes, coupons, fare type, tour code, plus the computed commission and outcome.
statement_row	One row per line of an uploaded consolidator statement: ticket number, amount paid, fees withheld, statement date.

4 · The pre-fill rule

The module returns a prefill field that is either a decimal string or null. **Bind it directly to the commission field.** The decision about whether a number is safe to write has already been made — the caller does not repeat it.

Outcome	prefill	Meaning
CALCULATED	the amount	A clause matched and produced a figure.
NIL	0.00	A clause asserted that nothing is due. A settled answer.
NO_RULE	null	A contract exists but no clause covers this ticket.
AMBIGUOUS	null	Two clauses matched equally well. Never guessed.
INCOMPLETE	null	A clause needed something the document did not carry.
NO_CONTRACT	null	No contract for this IATA. This is the fallback.
ERROR	null	The document could not be priced at all.

Two further conditions suppress a pre-fill even on a good outcome, and both were learned from real tickets:

Condition	Why
The layer being paid must be the layer that settled	On a Zürich–Tel Aviv reissue the carrier layer resolved cleanly (a \$100 clawback) while the sub-agent layer could not resolve its share at all. Reading one and paying the other offered a confident \$0.00 on an open question.
No REVIEW flag on the document	The engine raises REVIEW when two signed contracts disagree — on that same ticket, that the host pays \$100 out of pocket. A figure already flagged for a human is not one to bind to a form field.

Every result also carries explanation, clause and ruleId. Show the explanation next to the field. A number the agent cannot trace back to a line of the contract is a number they cannot argue with the consolidator about, which defeats the point of the module.

Fees are separate from commission

subAgentCommission is what was earned. netToSubAgent is what is actually received after the consolidator's fees. On a net or bulk fare these differ on every ticket — the commission reads 0.00 while the ticket costs \$15 to \$50. **Show the net.** Each fee line carries the clause it came from.

5 · Weekly statement reconciliation

The statement is uploaded as CSV or Excel. Each row is matched to a ticket by ticket number, and every row lands in exactly one of eight states. Nothing is dropped: a row that matches no ticket, and a ticket that appears on no row, are both findings.

Status	Severity	Meaning	Action
SHORT_PAID	critical	Paid less than the contract gives	Claim the difference
NOT_ON_STATEMENT	critical	Earned, but the statement omits it entirely	Claim in full
OVER_PAID	warning	Paid more than the contract gives	Expect a clawback
PAID_WHERE_NONE_DUE	warning	Paid on a document owed nothing	Verify before spending
NOT_IN_TICKETS	warning	Paid for a document not in our records	Investigate
DEDUCTION	warning	A fee withheld that no supplied agreement covers	Query the fee
AGREES	ok	Matches to the cent	—
CORRECTLY_NIL	ok	Both sides agree nothing is due	—

The two critical states are the ones that cost money and are invisible without this module: a short payment and an omission both look like a normal week on a statement.

Excel export — column layout

One row per ticket, sorted **worst first** (critical, then warning, then ok) so the rows that need attention are at the top and everything below can be ignored. Variance is the money column: negative means underpaid.

Column	Example	Column	Example
Status	SHORT_PAID	Expected	70.00
Ticket	114-7503646565	Paid	55.04
Passenger	COHEN/EDMOUND	Variance	-14.96
Type	TKT	Fees withheld	0.00
Issue date	2026-03-02	Net expected	70.00

Column	Example	Column	Example
Route	JFK–TLV	Contract	EL AL 2026
Class	S	Clause	Attachment A
Fare type	published	Why	S files at 7%, 8% was claimed

Add a totals row: sum of variance, split into *short-paid*, *over-paid*, *missing* and *unexplained deductions*. That single figure is what the week is worth arguing about, and it is what the agent takes to the consolidator.

Conditional formatting. Red fill on the Variance cell where it is negative, amber where positive, none where zero. Freeze the header row. The point is that the agent should be able to open the file and see the week's exposure without reading a single number carefully.

6 · API surface

Three functions. All take strings and return plain JSON — no bigint, no Date, no class instances — so the whole response survives JSON.stringify into a database column or an HTTP body. Money is always a decimal string.

```
import { priceAirFile, priceAirFiles, checkStatement }
  from '@commission/sdk'

// One AIR file -> what each passenger's ticket is worth
const result = priceAirFile(airText, {
  config,           // contracts, loaded from your DB
  view: 'subagent', // or 'host'
})

for (const doc of result.documents) {
  doc.ticketNumber      // '114-7503646565'
  doc.prefill           // '60.00' or null -> leave the field empty
  doc.outcome           // 'CALCULATED' | 'NIL' | 'NO_RULE' | ...
  doc.subAgentCommission // '60.00' what was earned
  doc.netToSubAgent      // '-15.00' what is actually received
  doc.fees               // [{ clause, label, amount }]
  doc.explanation         // show this next to the field
  doc.flags              // [{ code: 'REVIEW', message }]
}

// A whole folder at once
priceAirFiles([{ name, text }, ...], opts)

// Weekly reconciliation
checkStatement(statementCsv, airFiles, opts)
```

An HTTP wrapper is also provided (POST /price, POST /statement, GET /health) with no dependencies, for callers that cannot import JavaScript.

Where the config comes from

config is a plain JSON object built from the three contract tables in §3. Load it per tenant, cache it, and rebuild it whenever a contract is edited. It has no behaviour of its own — it is data — which is what keeps contracts editable without a deploy.

Signed fee schedules

Most host agreements are one line: *we keep a point*. That is `retains_points` and needs nothing further. A real signed schedule is richer — a percentage on published fares, a flat fee by cabin on net fares, a charge per exchange and per refund, and a charge on tickets that earn nothing. Where one has been encoded, name it in `fee_schedule`; otherwise the plain retention applies, which is what any new agency starts from.

Clauses that cannot be decided from a ticket are left switched off. Corporate bookings, promotional net fares and discretionary minimum fees depend on facts an AIR file does not carry. They are encoded but marked unapproved, so the engine surfaces them as exposure rather than booking them as certainty. Keep that flag in the schema: `approved: boolean`.

7 · Build order

CRM first, standalone second. The CRM has real tickets arriving every day, which is the only way to find the record shapes that break a parser — and every serious bug so far has come from a real file, not from a test. A standalone product built first would be a guess about what other agencies need, validated by nobody.

Phase	Scope	Done when
1	Contract tables + resolution chain (§2, §3). Read-only: compute and display, write nothing.	Every ticket in the CRM shows an outcome and an explanation.
2	Pre-fill (§4). Write the number into the commission field.	Agents stop entering commission by hand on CALCULATED tickets.
3	Statement upload + reconciliation + Excel export (§5).	A week's statement produces a sorted exception list.
4	Contract editing and PDF upload in the UI.	A new airline contract can be added without a deploy.
5	Extract as standalone for other agencies.	The engine already has no CRM dependency — this is packaging, not a rewrite.

Phase 1 being read-only is deliberate. It lets the computed figure be compared against what agents actually entered, across thousands of historical tickets, before a single number is written automatically. That comparison is the cheapest validation available.

Non-negotiables

Rule	Reason
Money is integer minor units, never floats	A half-cent bias is invisible per ticket and material across a year.
NIL and NO_RULE never merge	Both show \$0.00; only one is an answer.

Rule	Reason
A tie is never broken arbitrarily	Two clauses matching equally returns AMBIGUOUS. A wrong confident number costs more than a queued one.
Every figure cites its clause	The output is evidence in a dispute, not just a number.
The engine does no I/O	No database, no clock, no randomness — the same ticket always prices the same.
Unapproved clauses do not spend	A term nobody has confirmed is exposure, not income.

Open questions for the consolidator

These are money, and each needs an answer rather than a guess. They are recorded in the code alongside the clause they come from.

Question	Impact
Is the commissionable basis the base fare alone, or base plus YQ/YR? Neither contract states it.	Roughly \$35 per ticket on a typical long-haul fare
Which booking classes count as Economy, Premium and Business for the net-fare fee?	Decides a \$15 fee from a \$50 one on every net fare
Is the tour code enforced in practice, or only on paper?	Currently forfeits the commission on every sampled ticket
Is the minimum fee on small commissions applied automatically or at discretion?	\$10 per affected ticket
Statements are weekly but settlement is monthly — which is authoritative?	Decides whether a weekly shortfall is real or just timing